

机器学习实验指导手册

概念学习

一、实验目的

1. 理解概念学习的基本原理与假设空间表示方法
2. 掌握 Find-S 算法的实现与应用
3. 掌握候选消除算法的实现与变型空间维护
4. 通过实验分析不同算法在概念学习中的表现差异

二、背景知识

1. 概念学习框架

- 实例空间(X): 所有可能的观测样本集合
- 假设空间(H): 所有可能假设的集合
- 目标概念(C): $X \rightarrow \{0,1\}$ 的布尔函数
- 训练样例(D): $\langle x, c(x) \rangle$ 的有序对集合

2. Find-S 算法

- 基本思想: 从最特殊假设开始, 逐步泛化
- 特点:

- 只处理正例
- 保持假设与所有正例一致
- 输出最具体的满足条件的假设

3. 候选消除算法

- 核心机制：维护两个边界集合
 - S 集合：最具体假设的集合
 - G 集合：最一般假设的集合
- 变型空间：S 与 G 之间所有与训练样例一致的假设

三、实验内容

任务 1：实现 Find-S 算法

要求：

1. 理解 `agree()` 函数的实现逻辑
2. 完成 `find_S()` 函数中缺失的泛化处理
3. 测试不同训练样例顺序对结果的影响

示例代码片段：

```
```python
```

```
def find_S(examples):
 h = ['$']*len(attributes) # 初始化最特殊假设
 for e in examples:
 if not e[1]: # 跳过负例
 continue
 if not all(h[i] in (e[0][i], '$', '?') for i in range(len
(h))):
```

```

 h = [e[0][i] if h[i] == '$' else '?' for i in range(len
(h))]

 return h

```

---

## 任务 2：实现候选消除算法

### 关键步骤：

1. 初始化 S 集合为最特殊假设，G 集合为最一般假设
2. 对每个训练样例：
  - 正例处理：泛化 S，剪枝 G
  - 负例处理：特化 G，剪枝 S
3. 维护变型空间的一致性

### 核心函数解析：

```

def generate_SG(examples):
 S, G = [empty_h], [full_h]

 for e in examples:
 if e[1]: # 正例处理
 G = [h for h in G if agree(h, e[0], True)]
 S = [min_generalize(s, e[0]) for s in S]
 S = [s for s in S if any(is_general_than(g, s) for g in
G)]

 else: # 负例处理
 S = [s for s in S if agree(s, e[0], False)]
 new_G = []
 for g in G:
 if not agree(g, e[0], False):
 new_G.extend(min_specialize(g))

 G = [g for g in new_G if any(is_special_than(g, s) for s
in S)]

```

```
return S, G
```

---

## 四、实验要求

### 基础要求

1. 实现 Find-S 算法并验证输出；
2. 实现候选消除算法，并验证输出，要求：
  - 能正确处理包含噪声的数据
  - 可视化显示变型空间的演化过程
  - 对比 S 集合和 G 集合的变化

### 扩展要求

3. 添加新的训练样例：

```
[['Sunny', 'Cold', 'Normal', 'Strong', 'Warm', 'Same'], False]
```

观察变型空间如何收缩

4. 分析算法时间复杂度与属性数量的关系
- 

## 五、关键函数说明与功能解析

### 1. agree(h, e, elabel)

功能：判断假设 h 是否与样例 <e, elabel> 一致

参数：

- h (list): 当前假设，形式为属性值的列表（如 ['Sunny', '?', 'Normal', ...]）
- e (list): 样例输入，形式为属性值的列表（如 ['Sunny', 'Warm', ...]）
- elabel (bool): 样例标签（True 表示正例，False 表示负例）

输出：

- bool: 若 elabel=True，当 h 覆盖 e 时返回 True；若 elabel=False，当 h 不覆盖 e 时返回 True
-

## 2. is\_special\_than(h1, h2, strict=False)

功能: 判断假设 h1 是否比 h2 **更特殊**

参数:

- h1, h2 (list): 待比较的两个假设
- strict (bool): 是否严格比较 (若为 True, 则完全相同返回 False)

输出:

- bool: 当 h1 的每个属性值等于 h2 或 h2 为 '?' 时返回 True
- 

## 3. is\_general\_than(h1, h2, strict=False)

功能: 判断假设 h1 是否比 h2 **更一般**

参数: 同 is\_special\_than

输出: - bool: 当 h1 的每个属性值等于 h2 或 h1 为 '?' 时返回 True

---

## 4. min\_generalize(h, e)

功能: 将假设 h **极小一般化**以覆盖正例 e

参数:

- h (list): 原假设 (如 ['Sunny', '\$', ...])
- e (list): 正例输入 (如 ['Sunny', 'Warm', ...])

输出:

- list: 泛化后的新假设 (例如将 '\$' 替换为 e 的属性值或 '?')
- 

## 5. min\_specialize(h)

功能: 生成假设 h 的**极小特殊化**候选集

参数:

- h (list): 原假设 (如 ['?', 'Warm', ...])

输出:

- generator: 生成所有可能的特化假设 (例如将 '?' 替换为具体属性值)
- 

## 6. generate\_SG(examples)

功能: 生成候选消除算法的 **S 集合** (最特殊边界) 和 **G 集合** (最一般边界)

参数:

- examples (list): 训练样例列表，每个样例格式为 [特征列表，标签]

输出：

- tuple (S, G): 更新后的 S 和 G 集合

---

## 7. generate\_VS(S, G)

功能：生成变型空间（Version Space），即所有与数据一致的假设

参数：

- S (list): 最特殊假设集合 - G (list): 最一般假设集合

输出：

- list: 变型空间中的假设列表（格式为 [[属性值], ...]）

---

## 8. find\_S(examples)

功能：实现 **Find-S** 算法，寻找最特殊假设

参数：

- examples (list): 训练样例列表（仅处理正例）

输出：

- list: 最特殊假设（如 ['Sunny', 'Warm', '?', ...]）

---

## 9. 函数调用关系图

generate\_SG

└─ agree # 判断假设与样例一致性

└─ min\_generalize # 正例处理：泛化 S 集合

└─ min\_specialize # 负例处理：特化 G 集合

find\_S

└─ agree # 验证假设是否覆盖正例

# 六、实验分析

## 测试案例

attributes = [

```
 ['Sunny', 'Rainy'],
 ['Warm', 'Cold'],
 ['Normal', 'High'],
 ['Strong', 'Light'],
 ['Warm', 'Cool'],
 ['Same', 'Change']
]
```

```
examples = [
 [['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same'], True],
 [['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same'], True],
 [['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change'], False],
 [['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change'], True]
]
```

## 预期结果

### Find-S 输出:

```
['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

候选消除算法输出（含变型空间）：

```
S: [['Sunny', 'Warm', '?', 'Strong', '?', '?']]
```

```
['Sunny', 'Warm', '?', '?', '?', '?']
```

```
['Sunny', '?', '?', 'Strong', '?', '?']
```

```
['?', 'Warm', '?', 'Strong', '?', '?']
```

```
G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?',
'?']]
```

---